

Shop Billing System

Python Console-Based Application — Project Report

Submitted by
Hardik Sharma

Registration Number
26BAI10723

Language Used: Python 3
Type: Command Line Interface (CLI) Application

1. Introduction

The Shop Billing System is a simple, console-based application written in Python that helps a shopkeeper generate a bill for items purchased by a customer. The program takes the number of items being billed, collects the name, quantity, and price of each item, calculates the total cost per item and the grand total, displays a formatted bill on the screen, and finally saves a copy of the bill as a text file with a timestamped filename inside a local 'bills' folder.

2. Objectives

- To build a simple and practical billing tool using core Python concepts.
- To take structured user input and validate it (numbers and non-negative amounts).
- To perform basic arithmetic operations (quantity × price) for each item.
- To display billing information in a clean, tabular, readable format.
- To permanently store each generated bill as a text file for record-keeping.

3. Tools & Technologies Used

Component	Details
Programming Language	Python 3
Built-in Modules	os, datetime
Interface Type	Command Line Interface (CLI)
Output	Console display + saved .txt bill file

4. Working / Methodology

The program follows a straightforward, linear flow of execution:

- 1 The program starts and prints a heading for the billing session.
- 2 It asks the user how many items are to be billed, using a validation function (get_int) that keeps prompting until a proper whole number is entered.
- 3 If the number of items is zero, the program prints a message and exits early.
- 4 For each item, the user is asked to enter the item name (defaults to 'Item_<n>' if left blank), the quantity (validated integer), and the price (validated non-negative float using get_float).
- 5 The total cost for each item (quantity × price) is calculated and stored.
- 6 Once all items are entered, the program prints a formatted bill table showing name, quantity, price, and total for every item, followed by the grand total.
- 7 A 'bills' folder is created (if it does not already exist) using os.makedirs.

- 8 A uniquely named text file is generated using the current date and time (bill_YYYYMMDD_HHMMSS.txt) via the datetime module.
- 9 The bill details are written into this text file, and a confirmation message with the filename is shown to the user.

5. Key Functions

Function	Purpose
get_int(text)	Repeatedly prompts the user until a valid whole number (digit string) is entered.
get_float(text)	Repeatedly prompts the user until a valid, non-negative decimal number is entered.
main()	Controls the overall flow: input collection, bill calculation, display, and saving to file.

6. Sample Output

A typical run of the program produces output similar to the following:

```
===== SHOP BILLING =====
How many items? : 2

Item 1
Name: Notebook
Qty: 3
Price: 40

Item 2
Name: Pen
Qty: 5
Price: 10

----- BILL -----
Name      Qty   Price  Total
Notebook    3   40.0   120.0
Pen         5   10.0    50.0
-----
Grand Total = 170.0
-----
Bill saved: bill_20260926_143210.txt
```

7. Features

- Input validation for quantities (integers) and prices (non-negative floats).
- Default auto-naming of items if the user leaves the name field blank.
- Automatic calculation of per-item and grand totals.
- Neatly formatted, aligned console output.
- Automatic creation of a 'bills' directory for storing generated bills.
- Unique, timestamp-based filenames prevent bills from overwriting each other.

8. Limitations & Future Scope

Limitations:

- No graphical user interface (GUI); the program runs only in the console.
- Bill data is not stored in a structured/searchable format such as a database or CSV.
- No support for discounts, taxes (e.g. GST), or multiple currencies.
- No option to edit or delete an item after it has been entered.

Future Scope:

- Add a GUI using Tkinter or a web interface using Flask/Django.
- Store bills in a database (SQLite/MySQL) for easy retrieval and reporting.
- Add tax and discount calculation support.
- Generate the bill directly as a PDF receipt instead of a plain text file.
- Add inventory/stock management linked to billing.

9. Conclusion

The Shop Billing System successfully demonstrates the use of fundamental Python concepts such as functions, loops, input validation, file handling, and the os and datetime modules to build a practical, real-world utility. It automates the process of billing, reduces manual calculation errors, and keeps a persistent record of every transaction, making it a useful mini-project for understanding core programming and file-handling concepts in Python.